

Transformer Architectures: Encoder, Decoder, Encoder–Decoder

(বিশেষ ফোকাস: Encoder আর্কিটেকচার)

আগের অংশগুলোতে আমরা Transformer-এর সাধারণ কাঠামো দেখেছি এবং বুঝেছি কীভাবে Transformer বিভিন্ন NLP টাস্ক সমাধান করে। এখন সময় এসেছে **Transformer**-এর তিনটি প্রধান আর্কিটেকচার গভীরভাবে বোঝার:

1. **Encoder-only**
2. **Decoder-only**
3. **Encoder–Decoder (Sequence-to-Sequence)**

এই তিনটির পার্থক্য বুঝতে পারলে তুমি খুব সহজেই সিদ্ধান্ত নিতে পারবে—

কোন টাস্কের জন্য কোন ধরনের Transformer মডেল সবচেয়ে উপযুক্ত।

এই অংশে আমরা মূলত **Encoder architecture**—এর উপর জোর দেব, কারণ তোমার দেওয়া দুইটা কনটেন্টই সেটার দিকেই ফোকাস করেছে।

১) Encoder Architecture কী?

Encoder-only Transformer মানে হলো—

- Transformer-এর শুধু **encoder** অংশ ব্যবহার করা
- কোনো decoder থাকে না

সবচেয়ে পরিচিত উদাহরণ:

- **BERT**
- **DistilBERT**
- **ModernBERT**

Encoder-এর মূল কাজ:

ইনপুট বাক্যকে গভীরভাবে “বোঝা” এবং প্রতিটি শব্দের জন্য অর্থবহ representation তৈরি করা।

1) Encoder কীভাবে কাজ করে? (ভিতর থেকে বোঝা)

ইনপুট → আউটপুট ক্লে

Encoder মডেল ইনপুট হিসেবে নেয়:

- শব্দ বা token

আউটপুট হিসেবে দেয়:

- প্রতিটি token-এর জন্য একটি সংখ্যাগত ভেক্টর (**numerical representation**)

এই ভেক্টরগুলোকে বলা হয়:

- feature vector
- embedding
- tensor (গাণিতিকভাবে)

ভেক্টরের আকার (**Dimensionality**)

একটি স্ট্যান্ডার্ড **BERT-base** মডেলের ক্ষেত্রে:

- প্রতিটি token-এর ভেক্টরের দৈর্ঘ্য = **768**

এই 768-ডাইমেনশনের ভেক্টরের ভিতরেই থাকে:

- শব্দের অর্থ
- বাক্যের প্রেক্ষাপট
- আশেপাশের শব্দের প্রভাব

2) Encoder-এর সবচেয়ে বড় শক্তি: **Bidirectional Context**

Encoder মডেলগুলোকে বলা হয়:

Bidirectional বা **Auto-encoding models**

এর মানে:

- একটি শব্দ বোঝার সময় encoder দেখে:
 - বাম পাশের শব্দগুলো
 - ডান পাশের শব্দগুলো

উদাহরণ:

I went to the bank to deposit money

এখানে “bank” শব্দটা বোঝার জন্য encoder:

- “deposit”
 - “money”
- এই শব্দগুলোর দিকেও তাকায়

আবার:

I sat by the river bank

এখানে “bank” এর representation একেবারেই আলাদা হবে।

এই ক্ষমতাকে বলা হয়:

Contextualization

3) Self-Attention: Encoder-এর মূল ইঞ্জিন

Encoder এই bidirectional বোঝাপড়াটা করতে পারে কারণ এর ভেতরে থাকে:

Self-Attention Mechanism

Self-attention মানে:

- বাক্যের প্রতিটি শব্দ
- বাক্যের অন্য সব শব্দের দিকে তাকায়
- এবং সিদ্ধান্ত নেয়:
 - কার সাথে তার সম্পর্ক বেশি
 - কার প্রভাব বেশি

ফলে:

- একটি শব্দের representation কখনোই একা তৈরি হয় না
- পুরো বাক্য মিলেই প্রতিটা শব্দের অর্থ নির্ধারণ করে

এটাই Encoder-কে এত শক্তিশালী করে।

4) Encoder মডেল কীভাবে Pretrain করা হয়?

Encoder-only মডেলগুলো সাধারণত **self-supervised learning** দিয়ে pretrain করা হয়।

সবচেয়ে জনপ্রিয় পদ্ধতি:

Masked Language Modeling (MLM)

এখানে কী হয়:

- বাক্যের কিছু শব্দ ইচ্ছাকৃতভাবে লুকিয়ে (mask) দেওয়া হয়
- মডেলকে বলা হয়:
“চারপাশ দেখে বলো, এখানে কোন শব্দটা থাকার কথা”

উদাহরণ:

I love [MASK] learning

Encoder দেখে:

- “I love”
- “learning”

এবং প্রেডিক্ট করে:

- “machine” বা “deep”

এখানে encoder পুরো বাক্যের দুই দিকের তথ্য ব্যবহার করতে পারে বলেই MLM সম্ভব।

5) Encoder মডেল কোথায় সবচেয়ে ভালো কাজ করে?

Encoder মডেলগুলো **sequence** থেকে অর্থ বের করতে অসাধারণ দক্ষ।

প্রধান Use Cases

১) Sequence / Text Classification

পুরো বাক্য বা ডকুমেন্টের জন্য একটি লেবেল দেওয়া।

উদাহরণ:

- Sentiment analysis (positive / negative)
- Topic classification
- Spam detection

Encoder পুরো বাক্য বুঝে সিদ্ধান্ত নেয়।

২) Token / Word Classification

প্রতিটি শব্দের জন্য আলাদা লেবেল দেওয়া।

উদাহরণ:

- Named Entity Recognition (NER)
- Part-of-speech tagging

এখানে encoder-এর contextual representation খুব গুরুত্বপূর্ণ।

৩) Question Answering (Extractive)

- প্রশ্ন + কনটেক্সট দেওয়া হয়
- Encoder কনটেক্সট বুঝে
- কনটেক্সটের ভেতর থেকে উত্তর বের করে

Encoder এখানে “বোঝার” কাজটাই করে, নতুন করে লেখা তৈরি করে না।

6) Encoder কেন Text Generation-এর জন্য ভালো না?

এটা খুব গুরুত্বপূর্ণ ধারণা।

Encoder:

- পুরো বাক্য একসাথে দেখে
- ভবিষ্যৎ শব্দও দেখতে পায়

কিন্তু text generation-এর জন্য দরকার:

- বাম থেকে ডানে ধাপে ধাপে লেখা
- ভবিষ্যৎ শব্দ না দেখা

এই কারণেই:

- Encoder-only মডেল text generation-এর জন্য উপযুক্ত না
 - Text generation-এর জন্য decoder-only বা encoder-decoder দরকার
-

7) Encoder Architecture এক কথায়

সংক্ষেপে বললে:

- Encoder = Language Understanding Expert
 - এটি শব্দ, বাক্য, কনটেন্ট গভীরভাবে বোঝে
 - Bidirectional attention ব্যবহার করে
 - Pretraining হয় Masked Language Modeling দিয়ে
 - সবচেয়ে ভালো কাজ করে:
 - classification
 - NER
 - extractive QA
-

8) কেন Encoder Architecture বোঝা এত জরুরি?

কারণ বাস্তবে:

- ৭০-৮০% NLP সমস্যা হলো “বোঝার সমস্যা”
- সেখানে text generate করার দরকার নেই
- শুধু সঠিক অর্থ, সম্পর্ক, লেবেল দরকার

এই জায়গাতেই:

Encoder-only Transformer মডেলগুলো সবচেয়ে শক্তিশালী অস্ত্র

ঠিক আছে। এখন আমি তোমার দেওয়া দুইটা কনটেন্ট (ডকুমেন্টের অংশ + ভিডিও সারাংশ) একসাথে “শিখে” নিয়ে, নিজের ভাষায় আরও পরিষ্কারভাবে, ডিটেইলসে, শিক্ষক-ভঙ্গিতে **Decoder-only Transformer architecture** ব্যাখ্যা করছি। লক্ষ্য হলো তুমি বুঝবে: ডিকোডার কী, কেন এটা **text generation**-এ সেরা, **masked self-attention** কীভাবে কাজ করে, আর কোন কাজে এটা কম উপযোগী।

২) Decoder Models (Decoder-only Transformer):

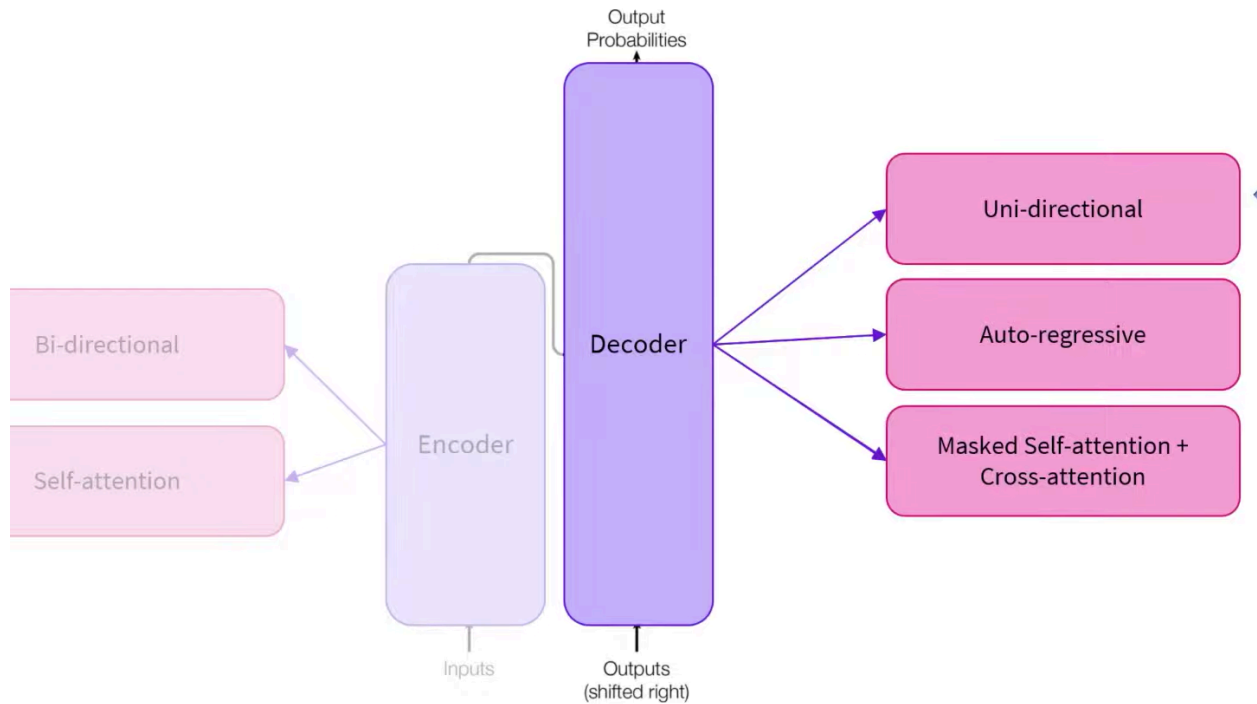
Transformer-এর তিনটা প্রধান আর্কিটেকচারের মধ্যে Decoder-only হলো এমন মডেল, যেখানে শুধু **decoder** অংশ থাকে—কোনো encoder থাকে না। এই পরিবারকে সাধারণভাবে বলা হয়:

- **Auto-regressive models**

কারণ তারা টেক্সট তৈরি করে একটা **token** করে, আগের **token** গুলোর উপর ভিত্তি করে।

সবচেয়ে পরিচিত উদাহরণ:

- GPT সিরিজ (GPT-2, GPT-3 টাইপ ধারণা)
- Meta Llama সিরিজ
- Google Gemma সিরিজ
- Hugging Face SmoLLM সিরিজ
- DeepSeek V3 (এই পরিবারে উল্লেখ করা হয়েছে)



১) **Decoder** মডেল আসলে কী করে?

Decoder মডেল ইনপুট হিসেবে যে টোকেনগুলো (শব্দ/সাবওয়ার্ড) পায়, তাদের প্রতিটির জন্য তৈরি করে:

- একটি **numerical representation / feature vector**

যেমন ইনপুট:

“Welcome to Bangladesh”

মডেল প্রতিটি token-এর জন্য ভেক্টর দেবে:

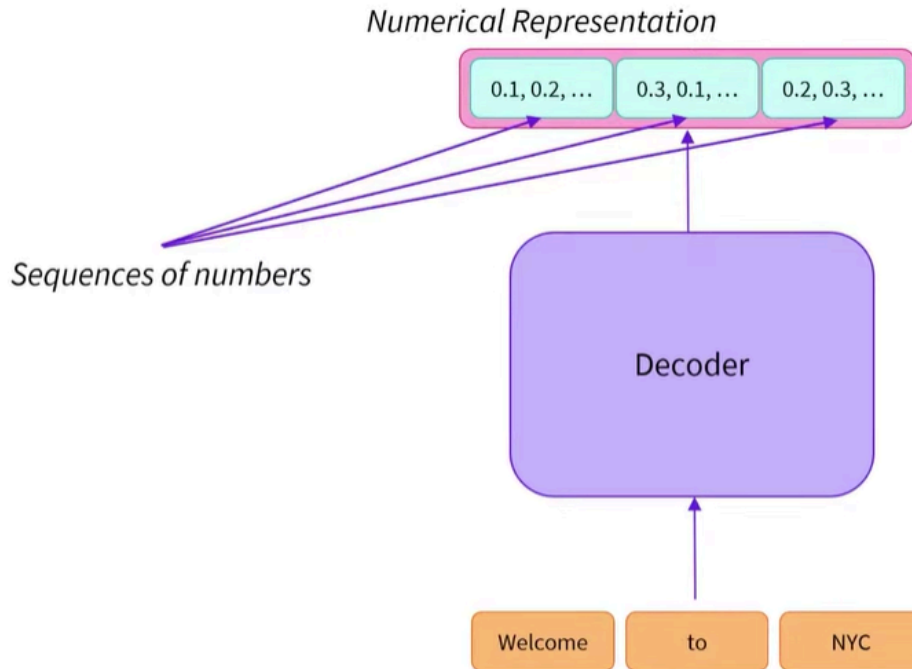
- Welcome → vector
- to → vector
- Bangladesh → vector

এই ভেক্টরের dimension মডেলের উপর নির্ভর করে। উদাহরণ হিসেবে:

- GPT-2 (base) এ সাধারণত hidden size 768 (তোমার সোর্সেও এটিই বলা)

খেয়াল করো:

Encoder যেমন representation তৈরি করে “বোঝার জন্য”, decoder-ও representation বানায়—কিন্তু decoder-এর representation তৈরি হয় একটা নির্দিষ্ট সীমাবদ্ধ নিয়ম মেনে।



২) Decoder বনাম Encoder: সবচেয়ে বড় পার্থক্য কোথায়?

Decoder আর Encoder দেখতে অনেকটা একইরকম হলেও সবচেয়ে বড় পার্থক্য হলো:

Masked Self-Attention

Decoder-এ self-attention “masked”, মানে:

- কোনো token যখন প্রসেস হচ্ছে, সে ডান পাশের **future token** দেখতে পারবে না
- সে শুধুমাত্র বাম পাশের আগের **token** গুলো দেখতে পারবে

এই জন্য decoder কে বলা হয়:

- **Causal** (কারণ ভবিষ্যৎ কারণ নয়, অতীত কারণ)

- **Left-to-right attention**

সহজ উদাহরণ

ইনপুট:

My name is Sakhawat

যখন মডেল “name” token প্রসেস করছে, তখন:

- “My” দেখতে পারবে
- কিন্তু “is Sakhawat” দেখতে পারবে না

এখানেই masked attention কাজ করে:

future অংশগুলোর attention score কার্যত ব্লক (0) করে দেয়।

৩) কেন এই “ডান দিক না দেখা” নিয়মটা দরকার?

কারণ decoder-এর কাজ হলো টেক্সট **generate** করা।

টেক্সট জেনারেশনের বাস্তব পরিস্থিতিতে:

- তুমি আগে থেকে জানো না পরের শব্দ কী হবে
- তাই মডেলকেও “cheat” করতে দেওয়া যাবে না

যদি মডেল ভবিষ্যৎ শব্দ দেখতে পেত, তাহলে training খুব সহজ হয়ে যেত:

- সে আগে থেকেই উত্তর দেখে ফেলত

এই কারণেই decoder-এ:

- masked self-attention বাধ্যতামূলক

৪) Decoder মডেল কীভাবে Pretrain করা হয়?

Decoder-only মডেলগুলো সাধারণত pretrain হয়:

Next-token prediction (Causal Language Modeling)

অর্থাৎ লক্ষ্য:

- আগের tokenগুলো দেওয়া থাকবে

- মডেলকে পরের token অনুমান করতে হবে

উদাহরণ:

ইনপুট: I love

টার্গেট: machine

এটা বারবার করতে করতে মডেল শিখে ফেলে:

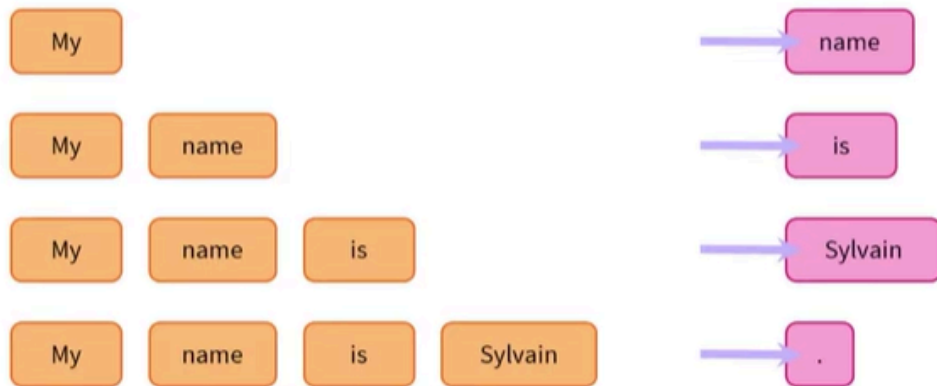
- ভাষার pattern
- বাক্যের গঠন
- কোন শব্দের পরে কোন শব্দ বেশি সম্ভব

এখান থেকেই আসে তাদের শক্তি:

Text generation

Causal Language Modeling

Guessing the next word in a sentence



৫) Auto-regressive Generation কীভাবে হয়? (ধাপে ধাপে)

এটা এমন:

1. শুরুতে prompt দাও: "My"
2. মডেল প্রেডিক্ট করল next token: "name"
3. এখন ইনপুট হলো: "My name"

4. আবার প্রেডিক্ট: "is"
5. ইনপুট হলো: "My name is"
6. আবার প্রেডিক্ট: "Sakhawat"
7. এভাবে চলতে থাকে...

অর্থাৎ:

- মডেল নিজের আগের আউটপুটকে আবার ইনপুট হিসেবে ব্যবহার করে
- এই loop-টাই auto-regressive generation

এখানে আরেকটা জিনিস কাজ করে:

- context window / max sequence length
GPT-2 প্রায় 1024 token পর্যন্ত কনটেক্সট ধরে রাখতে পারে (বাস্তবে বিভিন্ন ভ্যারিয়েন্টে ভিন্ন হতে পারে)।

৬) Decoder-only মডেল কোন কাজে সেরা?

Decoder-only মডেল সবচেয়ে ভালো:

Text generation-ভিত্তিক টাস্কে

যেমন:

- গল্প লেখা
- ব্লগ/আর্টিকেল ড্রাফট
- চ্যাট রেসপন্স
- কোড জেনারেশন
- ইমেইল/কন্টেন্ট অটো-কমপ্লিশন

কারণ:

- এগুলোর সবগুলোর মূল কাজ “পরের টোকেন তৈরি করা”

এখানেই decoder-এর causal training objective সরাসরি কাজে লাগে।

৭) Decoder-only মডেল কোন কাজে তুলনামূলক কম ভালো?

Decoder-only মডেল bidirectional নয়। সে ডান দিক দেখে না। তাই:

- Sentence classification

- Named entity recognition
 - Extractive question answering
- এ ধরনের “পূর্ণ বাক্য বুঝে সিদ্ধান্ত” টাস্কে Encoder-only (BERT টাইপ) সাধারণত বেশি উপযোগী।

তবে decoder দিয়ে এগুলো করা যায় না এমন নয়—কিন্তু:

- architecture match কম
- অনেক সময় training/data প্রয়োজন বেশি পড়ে
- বা prompt-based workaround লাগে

সোজা কথা:

- Decoder: লেখা তৈরি করতে দারুণ
- Encoder: লেখা বুঝতে দারুণ

৮) Encoder–Decoder এ গেলে cross-attention আসে কেন?

যখন decoder একটি encoder-এর সাথে যুক্ত হয়, তখন থাকে:

Cross-attention

এখানে decoder:

- নিজের আগের টোকেন (self-attention) তো দেখেই
- পাশাপাশি encoder-এর আউটপুট representation-ও দেখে

এটা দরকার হয় sequence-to-sequence টাস্কে:

- translation
- summarization
- speech-to-text (Whisper টাইপ)

এগুলোতে decoder শুধু “আগের token” দেখলে হবে না—ইনপুট সোর্সের context-ও লাগবে, যেটা encoder দেয়।

Modern Large Language Models (LLMs)

আজকের সময়ের বেশিরভাগ Large Language Model (LLM) তৈরি হয় decoder-only Transformer architecture দিয়ে।

Decoder-only মানে হলো:

- মডেলে শুধু **Transformer decoder** ব্লক থাকে
- মডেল টেক্সট প্রসেস করে **left-to-right** (বাম থেকে ডান দিকে)
- এবং প্রতিবার আগের **token** দেখে পরের **token** প্রেডিক্ট করে

এই কারণেই এদেরকে বলা হয়:

- **auto-regressive language model**

গত কয়েক বছরে LLM গুলোর সাইজ (**parameter**) ও ক্ষমতা (**capability**) খুব দ্রুত বেড়েছে। সবচেয়ে বড় মডেলগুলোতে থাকে শত শত বিলিয়ন **parameters**।

Parameter যত বেশি হয় (সঠিক ট্রেনিং/ডেটা/ডিজাইন থাকলে), মডেল তত বেশি জটিল ভাষার **pattern** ধরতে পারে—ফলে ক্ষমতা বাড়ে।

১) আধুনিক LLM কীভাবে ট্রেন করা হয়? (Two-phase training)

Modern LLM সাধারণত দুই ধাপে ট্রেন হয়:

Phase 1: Pretraining (ভাষার ভিত্তি শেখা)

এই ধাপে মডেলকে বিশাল পরিমাণ টেক্সট ডেটা দেওয়া হয় এবং শেখানো হয় একটি মূল কাজ:

“এই পর্যন্ত যা লেখা আছে, তার পরের token কী হতে পারে?”

এটাই **next-token prediction** বা **causal language modeling**।

এখানে ডেটা সাধারণত:

- ওয়েব টেক্সট
- বই
- আর্টিকেল
- কোড
- ডকুমেন্টেশন

এই ধাপে মডেল যা শেখে:

- ভাষার ব্যাকরণ/স্ট্রাকচার
- শব্দের সম্পর্ক (association)
- সাধারণ জ্ঞান (অনেকটা implicit ভাবে)
- “কোন প্রসঙ্গে কোন ধরনের কথা আসে” এই pattern

কিন্তু খেয়াল করো:

Pretraining শেষে মডেল ভাষা জানে, কিন্তু “ভদ্রভাবে নির্দেশ মানা” এখনো নিশ্চিত নয়।

Phase 2: Instruction Tuning (ইনস্ট্রাকশন মানা শেখানো)

এই ধাপে pretrained মডেলকে আবার fine-tune করা হয় এমন ডেটা দিয়ে যেখানে থাকে:

- instruction (নির্দেশ)
- expected helpful response (সঠিক/সহায়ক উত্তর)

উদাহরণ:

- “এই টেক্সটটা summarize করো” → ভালো summarization output
- “এই কোডটা ঠিক করো” → ঠিক করা কোড
- “আমাকে step-by-step বুঝাও” → ধাপে ধাপে ব্যাখ্যা

এই ধাপের ফলাফল:

- মডেল ব্যবহারকারীর নির্দেশ ভালোভাবে ফলো করতে শেখে
- উত্তরগুলো আরও সহায়ক, প্রাসঙ্গিক ও ব্যবহারযোগ্য হয়

এই দুই ধাপের কম্বিনেশনেই আজকের LLM গুলো “assistant-like” হয়ে ওঠে।

২) Modern LLM-এর প্রধান ক্ষমতাগুলো (Key capabilities)

Decoder-based আধুনিক LLM গুলো অনেক ধরনের কাজে ভালো পারফর্ম করে। নিচে প্রতিটা ক্ষমতা সহজভাবে ব্যাখ্যা করছি:

১) Text Generation

প্রম্পট দিয়ে দিলে ধারাবাহিক, প্রাসঙ্গিক লেখা তৈরি করে।

উদাহরণ:

- প্রবন্ধ/গল্প লেখা
- ইমেইল ড্রাফট
- প্রোডাক্ট বর্ণনা

কেন পারে: কারণ core training-ই next-token prediction।

২) Summarization

লম্বা লেখা ছোট করে মূল কথা রেখে সারাংশ তৈরি করতে পারে।

উদাহরণ:

- রিপোর্টের executive summary
- মিটিং নোটের সংক্ষিপ্ত রূপ

৩) Translation

এক ভাষা থেকে আরেক ভাষায় অর্থ ঠিক রেখে অনুবাদ করতে পারে।

উদাহরণ:

- English → Spanish

৪) Question Answering

তথ্যভিত্তিক প্রশ্নের উত্তর দেয়।

উদাহরণ:

- “France-এর রাজধানী কী?”

এখানে সতর্কতা:

- LLM কখনো ভুল তথ্যও আত্মবিশ্বাসের সাথে দিতে পারে (hallucination)
- তাই গুরুত্বপূর্ণ ক্ষেত্রে যাচাই দরকার

৫) Code Generation

বর্ণনা দেখে কোড লিখতে পারে, অথবা অসম্পূর্ণ কোড শেষ করতে পারে।

উদাহরণ:

- “একটা Python function লেখো যা list থেকে duplicate remove করবে”

৬) Reasoning

সমস্যা ধাপে ধাপে ভেঙে সমাধান দিতে পারে।

উদাহরণ:

- অংক

- লজিক্যাল পাজল
- debugging reasoning

বাস্তবে এটি অনেক সময় “pattern + structured thinking” মিশিয়ে কাজ করে—তাই সব ক্ষেত্রে সমান পারফেক্ট নয়, কিন্তু অনেক কাজে দারুণ সহায়তা করে।

৭) Few-shot Learning

প্রম্পটে ২-৩টা উদাহরণ দেখালে মডেল সেই প্যাটার্ন ধরে নতুন ইনপুটেও একই স্টাইলে কাজ করতে পারে।

উদাহরণ:

- তুমি ২টা উদাহরণ দিয়ে দেখালে কোন বাক্য positive/negative
- তারপর নতুন বাক্য দিলে মডেল একই নিয়মে classify করার চেষ্টা করে

এটা “ট্রেনিং নয়”—এটা prompt-এর context থেকে শেখা, যাকে বলে:

- **in-context learning**

৩) Browser থেকেই LLM ট্রাই করা

তুমি decoder-based LLM গুলো সরাসরি ব্রাউজারে পরীক্ষা করতে পারো:

- Hugging Face Hub-এর model repo page এ গিয়ে
- widget-এ prompt লিখে output দেখা যায়
- ডাউনলোড/কোড না লিখেও quick test করা সম্ভব

ক্লাসিক উদাহরণ:

- **GPT-2**

GPT-2 এখন শেখার জন্য খুব ভালো, কারণ:

- ছোট
- দ্রুত রান হয়
- text generation কীভাবে কাজ করে চোখের সামনে বোঝা যায়

৩ Sequence-to-sequence Models (Encoder–Decoder Transformers)

Sequence-to-sequence (সংক্ষেপে **Seq2Seq**) বা **Encoder–Decoder** মডেল হলো Transformer-এর এমন একটি আর্কিটেকচার যেখানে **Encoder** এবং **Decoder**—দুটো অংশই একসাথে থাকে। এটা মূলত ব্যবহার হয় এমন সব টাস্কে যেখানে:

একটি ইনপুট টেক্সট/সিকোয়েন্সকে “বোঝার” পর
তার ভিত্তিতে আরেকটি নতুন আউটপুট সিকোয়েন্স “তৈরি” করতে হয়।

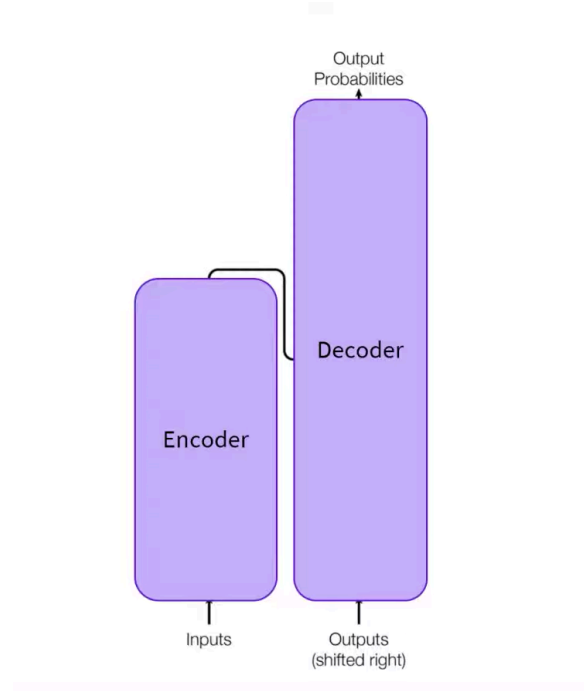
যেমন: Translation, Summarization, Generative QA, Grammar correction ইত্যাদি।

১) Encoder–Decoder আর্কিটেকচার আসলে কী?

Encoder–Decoder মডেলকে সহজভাবে ভাবতে পারো দুইজনের টিম হিসেবে:

- **Encoder** = বোঝার বিশেষজ্ঞ
ইনপুট টেক্সটটা পড়ে তার অর্থ, কনটেক্সট, সম্পর্ক—সব মিলিয়ে একটা গভীর representation বানায়।
- **Decoder** = লেখার/তৈরি করার বিশেষজ্ঞ
Encoder যে representation বানিয়েছে, সেটা দেখে ধাপে ধাপে আউটপুট লেখা তৈরি করে।

এখানে Encoder “উত্তর লিখে” না, Decoder “ইনপুট বোঝে” না—তারা আলাদা আলাদা কাজে স্পেশলাইজড।



২) Attention এখানে কীভাবে কাজ করে?

এটা বুঝলে Seq2Seq পুরোপুরি পরিষ্কার হয়ে যাবে।

Encoder-এর attention (Bidirectional)

Encoder যখন ইনপুট প্রসেস করে, তখন:

- বাক্যের সব শব্দই একসাথে দেখতে পারে
 - ডানে-বামে উভয় দিকের কনটেক্সট ব্যবহার করতে পারে
- এজন্য Encoder খুব ভালোভাবে ইনপুট “বোঝে”।

Decoder-এর attention (Masked / Causal)

Decoder যখন আউটপুট তৈরি করে, তখন:

- সে ভবিষ্যতের শব্দ দেখতে পারে না
 - শুধু আগের তৈরি করা শব্দগুলো দেখতে পারে
- এজন্য Decoder টেক্সট জেনারেশনে ভালো।

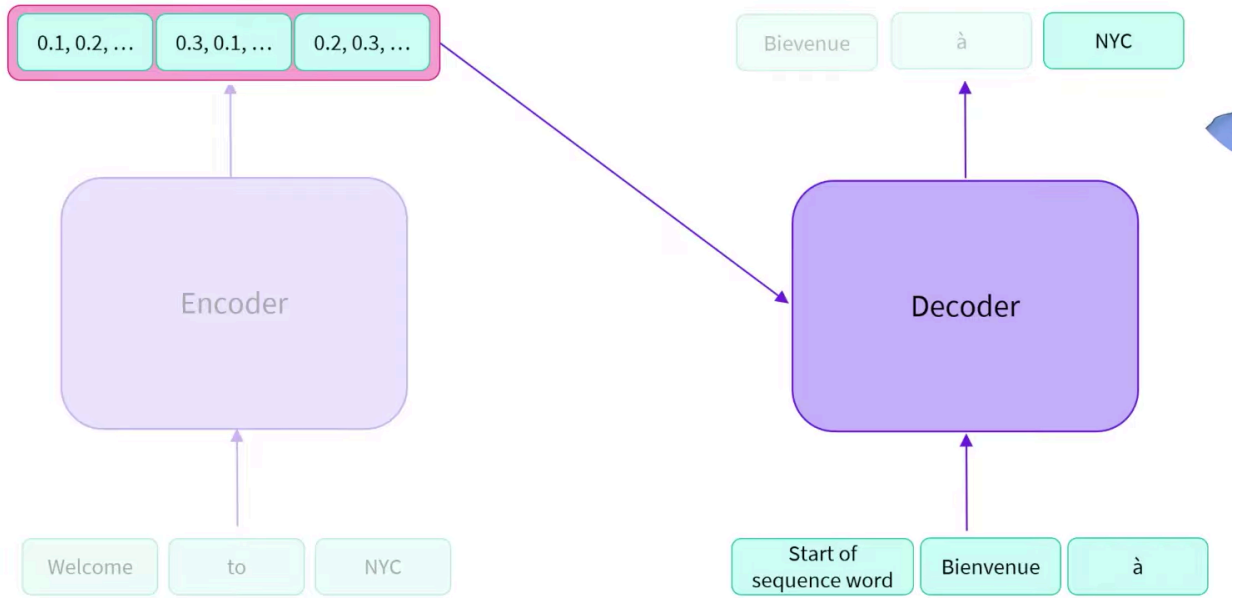
Cross-attention (Decoder থেকে Encoder-এর দিকে দেখা)

Decoder শুধু নিজের আগের শব্দ দেখে থেমে থাকলে সমস্যা—এতে ইনপুটের অর্থ হারিয়ে যাবে।
তাই Decoder আরেকটা attention ব্যবহার করে:

- Decoder, Encoder-এর তৈরি করা representation-এ তাকায়
- ইনপুটের কোন অংশ বেশি গুরুত্বপূর্ণ—সেটা বুঝে আউটপুট বানায়

এটাই Seq2Seq মডেলের “গেম-চেঞ্জার” ক্ষমতা।

৩) Encoder–Decoder কীভাবে কাজ করে? (পূর্ণ workflow)



ধরা যাক কাজ: Translation

ইনপুট: “**Welcome to NYC**”

আউটপুট: ফরাসি অনুবাদ (ধরি) “**Bienvenue à NYC**”

ধাপ ১: **Encoder** ইনপুট বোঝে

- Encoder পুরো বাক্যটা একবারে পড়ে
- প্রতিটি token-এর contextual representation বানায়
- এই representation-এর মধ্যে বাক্যের অর্থ, শব্দের সম্পর্ক—সব থাকে

ধাপ ২: **Decoder** লেখা শুরু করে

Decoder সাধারণত একটি বিশেষ টোকেন দিয়ে শুরু করে:

- Start of sequence (যেমন **<bos>**)

তারপর:

- Encoder-এর representation দেখে
- প্রথম আউটপুট token বানায় (যেমন “Bienvenue”)

ধাপ ৩: Auto-regressive generation

এখন Decoder যেটা বানিয়েছে, সেটাকেই আবার ইনপুট হিসেবে ব্যবহার করে:

- **<bos> Bienvenue**

এরপর আবার:

- Encoder-এর representation + আগের শব্দগুলো দেখে
- পরের শব্দ বানায় (“à”)

এভাবে চলতে থাকে:

- **<bos> Bienvenue à NYC ...**

ধাপ ৪: থামার সংকেত

যখন Decoder একটি end token (**<eos>**) বা থামার মানে এমন টোকেন তৈরি করে, তখন generation থেমে যায়।

এই পুরো প্রক্রিয়াকে বলা হয়:

- Encoder “বোঝে”
- Decoder “তৈরি করে”

৪) Pretraining কীভাবে হয়?

Encoder–Decoder মডেলগুলো pretrain করার উদ্দেশ্য সাধারণত এমন:

ইনপুটটা একটু “নষ্ট/বিকৃত” (corrupted) করে দাও
তারপর মডেলকে বলো মূল টেক্সটটা আবার “পুনর্গঠন” করো

T5-এর বিখ্যাত pretraining (Span corruption)

T5 মডেলে যা করা হয়:

- টেক্সটের কিছু অংশ (এক/একাধিক শব্দের span) তুলে দেওয়া হয়
- সেই জায়গায় একটি মাত্র mask token বসানো হয়
- তারপর মডেলকে বলা হয় ওই mask-এর আসল অংশটা প্রেডিক্ট করতে

উদাহরণ:

ইনপুট:

- “I love learning”

টার্গেট:

- “machine”

Span masking-এর সুবিধা:

- শুধু একটা শব্দ নয়, অনেক সময় পুরো বাক্যাংশ missing থাকে
- মডেল শিখে যায়:
 - কতটা টেক্সট অনুপস্থিত
 - কনটেক্সট থেকে কীভাবে পূরণ করতে হয়

এজন্য T5 translation, summarization—দুটোতেই শক্তিশালী।

৫) Seq2Seq মডেল কোথায় সবচেয়ে ভালো?

Seq2Seq মডেল সেরা হয় এমন কাজে যেখানে:

ইনপুট আছে, আর সেই ইনপুটের ভিত্তিতে নতুন আউটপুট তৈরি করতে হয়

যেমন:

- Translation
- Summarization
- Grammar correction
- Data-to-text
- Generative question answering

কারণ এখানে:

- Encoder ইনপুটের গভীর অর্থ ধরে
- Decoder সেই অর্থের ভিত্তিতে সুন্দর আউটপুট বানায়

৬) Practical applications

নীচের টেবিলটা যেমন ছিল, টেবিল আকারেই রাখা হলো:

Application	Description	Example Model
Machine translation	এক ভাষা থেকে আরেক ভাষায় অর্থ ঠিক রেখে অনুবাদ	Marian, T5
Text summarization	বড় লেখা ছোট করে মূল বক্তব্যসহ সারাংশ তৈরি	BART, T5
Data-to-text generation	টেবিল/স্ট্রাকচার্ড ডেটা থেকে মানুষ-ভাষায় লেখা তৈরি	T5
Grammar correction	ব্যাকরণগত ভুল ঠিক করে সঠিক বাক্য তৈরি	T5
Question answering	কনটেন্ট দেখে নতুন করে উত্তর তৈরি (generative)	BART, T5

৭) জনপ্রিয় Seq2Seq মডেলগুলো

এই পরিবারের গুরুত্বপূর্ণ মডেল:

- BART
- mBART
- Marian
- T5

৮) কোন টাস্কে কোন আর্কিটেকচার নেবে?

টেবিলটা আগের মতোই রাখা হলো:

Task	Suggested Architecture	Examples
Text classification (sentiment, topic)	Encoder	BERT, RoBERTa
Text generation (creative writing)	Decoder	GPT, LLaMA
Translation	Encoder-Decoder	T5, BART
Summarization	Encoder-Decoder	BART, T5
Named entity recognition	Encoder	BERT, RoBERTa
Question answering (extractive)	Encoder	BERT, RoBERTa

Question answering (generative)	Encoder-Decoder বা Decoder	T5, GPT
Conversational AI	Decoder	GPT, LLaMA

৯) শর্ট রুল

যদি কখনো দ্বিধায় পড়ো, এই তিনটা প্রশ্ন করো:

1. কাজটা কি “বোঝা” টাইপ? → Encoder
2. কাজটা কি “লেখা/জেনারেট” টাইপ? → Decoder
3. কাজটা কি “একটা লেখা থেকে আরেকটা লেখা বানানো”? → Encoder–Decoder (Seq2Seq)

LLMs-এর Evolution (বিবর্তন): কেন Attention এত গুরুত্বপূর্ণ?

LLM (Large Language Model) গুলো গত কয়েক বছরে খুব দ্রুত উন্নত হয়েছে—প্রতিটা নতুন “জেনারেশন”-এ মডেলগুলো আরও বড় হয়েছে, আরও ভালো কাজ করতে শিখেছে, এবং আরও বেশি বাস্তব সমস্যায় ব্যবহারযোগ্য হয়েছে। এই উন্নতির বড় একটা অংশ এসেছে **Attention mechanism**—এ নতুন নতুন ডিজাইন/অপটিমাইজেশন থেকে।

১) Attention mechanism: সমস্যাটা কোথায়?

Transformer-এর মূল শক্তি হলো **Self-Attention**:

একটা সিকোয়েন্সের প্রতিটি token অন্য সব token-এর সাথে সম্পর্ক ধরতে পারে।

কিন্তু এখানেই বড় সমস্যা:

Full attention কেন ব্যয়বহুল?

স্ট্যান্ডার্ড attention-এ attention matrix হয় $n \times n$ (square), যেখানে:

- n = sequence length (টোকেন সংখ্যা)

তাই হিসাব/কম্পিউটেশন লাগে:

- $O(n^2)$

মানে: sequence বড় হলেই খরচ হু হু করে বাড়ে।

উদাহরণ (ধারণা):

- 1,000 টোকেন $\rightarrow$ 1,000,000 (প্রায়) interaction
 - 10,000 টোকেন $\rightarrow$ 100,000,000 (প্রায়) interaction
- এটা GPU মেমোরি ও সময়—দুইদিক থেকেই bottleneck।

এই সীমাবদ্ধতার জন্য Longformer, Reformer-এর মতো মডেলগুলো এসেছে—যারা attention-কে **sparse** বা **efficient** করে দীর্ঘ টেক্সট হ্যান্ডেল করতে চায়।

২) Specialized attention কেন দরকার?

লম্বা ডকুমেন্ট/বই/কোডবেস/লিগ্যাল টেক্সট/রিসার্চ পেপার—এগুলোতে context অনেক বড় হয়।

Full attention দিয়ে এত বড় context চালাতে গেলে:

- GPU মেমোরি explode করতে পারে
- training/inference খুব ধীর হয়ে যায়

তাই লক্ষ্য হলো:

সব token-কে সব token-এর সাথে না জুড়ে, “প্রয়োজনীয় সংযোগগুলোই” রাখা

এভাবেই আসে **Sparse Attention** ধারণা।

৩) LSH Attention (Reformer): “যাদের দরকার, শুধু তাদেরই দেখো”

Reformer ব্যবহার করে **LSH attention** (Locality Sensitive Hashing)।

আইডিয়া

স্ট্যান্ডার্ড attention-এ $\text{softmax}(QK^T)$ হিসাব হয়। বাস্তবে দেখা যায়:

- QK^T ম্যাট্রিক্সের সব element সমান গুরুত্বপূর্ণ না
- বড় বড় কয়েকটা element-ই softmax -এর পরে সবচেয়ে বেশি প্রভাব ফেলে

সুতরাং ধারণা দাঁড়ায়:

প্রতিটা query q-এর জন্য সব key k দেখা লাগবে না
q-এর কাছাকাছি (similar) key গুলো দেখলেই হবে

“কাছাকাছি” মানে কীভাবে ধরবে?

এখানে আসে hash function:

- একটি hash function ব্যবহার করে q এবং k “একই bucket”-এ পড়লে তাদেরকে কাছাকাছি ধরা হয়
- তারপর attention শুধু সেই bucket-এর token গুলোর মধ্যেই চালানো হয় (সব token জুড়ে না)

কেন **multiple hash rounds (n_rounds)**?

Hash কিছুটা random হতে পারে। তাই:

- একাধিক hash function/round ব্যবহার করা হয়
- তারপর ফলাফলগুলো average করা হয়
এতে stability বাড়ে।

Masking কেন?

Reformer-এ attention mask এমনভাবে বদলানো হয় যাতে:

- token নিজেকে নিজে অতিরিক্ত সুবিধা না দেয়
কারণ একই token হলে query=key হওয়ায় similarity সর্বোচ্চ হয়ে যায়, যা learning-এ “shortcut” হতে পারে।
তাই current token (প্রথম পজিশন ছাড়া) masking করা হয়।

ফল: Full attention না চালিয়ে অনেক কম খরচে কাজ করা যায়।

8) Local Attention (Longformer): “একটা ছোট জানালার ভেতর দেখো + কিছু গ্লোবাল টোকেন”

Longformer ব্যবহার করে **Local attention**।

Local attention কী?

এখানে প্রতিটা token শুধু তার আশেপাশের একটি ছোট window দেখে, যেমন:

- বামে ২টা token
- ডানে ২টা token
(বাস্তবে window size বড়ও হতে পারে)

কারণ অনেক কাজেই (বিশেষ করে ভাষায়):

- কাছের শব্দগুলোর context-ই বেশিরভাগ সময় যথেষ্ট

তাহলে দূরের **context** যাবে কোথায়?

এখানে গুরুত্বপূর্ণ ধারণা হলো **layer stacking**:

- যদি একাধিক attention layer থাকে
- এবং প্রতিটা layer ছোট window দেখে
তাহলে উপরের layer-গুলোতে গিয়ে receptive field বড় হয়ে যায়
অর্থাৎ পরোক্ষভাবে দূরের token-এর তথ্যও পৌঁছাতে পারে।

Global attention (কিছু বিশেষ token)

Longformer আরও একটা কাজ করে:

- কিছু preselected token-কে **global attention** দেওয়া হয় যেমন:
- [CLS] টোকেন
- প্রশ্নের টোকেন (QA task)
- বিশেষ delimiter token ইত্যাদি

Global token-এর বৈশিষ্ট্য:

- তারা সব token দেখতে পারে
- এবং অন্য সব token-ও তাদের দেখতে পারে (সিমেট্রিক access)

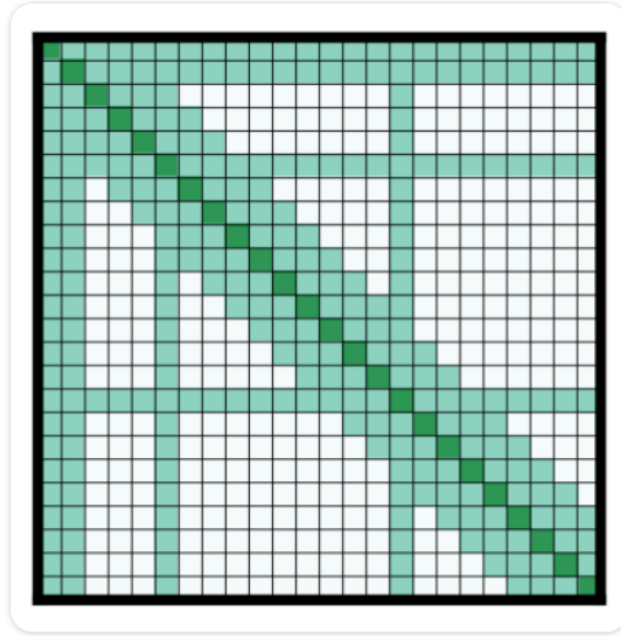
ফলে:

- লোকাল window-এর সীমাবদ্ধতা কাটে
- তবুও full attention-এর মতো n^2 খরচ হয় না

লাভ

Sparse/local attention ব্যবহার করে:

- কম parameter/কম compute-এ
- বড় sequence length (long context) চালানো সম্ভব হয়



৫) Axial Positional Encoding (Reformer): “বিশাল **positional matrix** ছোট করে ফেলো”

Transformer-এ positional encoding সাধারণত একটি ম্যাট্রিক্স:

- E এর সাইজ: $l \times d$
 - l = sequence length
 - d = hidden dimension

লম্বা sequence হলে:

- l অনেক বড়
- তাই E বিশাল হয়ে GPU-তে জায়গা বেশি নেয়

Axial encoding কী করে?

Reformer E ম্যাট্রিক্সকে factorize করে দুইটা ছোট ম্যাট্রিক্সে:

- $E1: l1 \times d1$
- $E2: l2 \times d2$

যেখানে:

- $l1 \times l2 = l$
- $d1 + d2 = d$

এতে মোট storage অনেক কমে যায়, কারণ বড় ম্যাট্রিক্সের বদলে দুইটা ছোট ম্যাট্রিক্স রাখা লাগে।

timestamp j-এর embedding কীভাবে তৈরি হয়?

timestamp j-এর জন্য embedding বানানো হয়:

- E1 থেকে index: $j \% l1$
 - E2 থেকে index: $j // l1$
- তারপর দুইটা embedding **concatenate** করে দেওয়া হয়।

ফল:

- memory efficient positional encoding
- খুব লম্বা sequence-এর ক্ষেত্রেও feasible

৬) এই সবকিছুর সারমর্ম (কেন এটা LLM evolution-এর অংশ)

LLM উন্নত হওয়ার বড় একটি বাধা ছিল:

- long context = expensive attention

Longformer / Reformer-এর মতো মডেলগুলো দেখিয়েছে:

- attention sparse করলে এবং positional encoding efficient করলে
- অনেক বড় sequence চালানো যায়
- training দ্রুত হয়, memory কম লাগে

এটা LLM evolution-এর একটা গুরুত্বপূর্ণ ধাপ:

- “বড় মডেল” শুধু parameter বাড়ালেই হয় না
- “দীর্ঘ কনটেক্সট” efficient ভাবে হ্যান্ডেল করাও জরুরি